

Homework 6: Code Optimization

Name: Eliana Kang

GWID: 25866679

Introduction

Goal: optimize the rotate and smooth functions to achieve maximum speedup using techniques we learned in lectures.

Testing environment:

- System: shell01.seas.gwu.edu
- Compiler: gcc (no compiler optimization options)
- Performance measurement: average of 10 runs
- L1 cache: 32768 bytes
- Cache line: $64/6=10.67$; each line holds about 10 pixels
- L2 cache: 1048576 bytes
- L3 cache: 20185088 bytes

```
[G25866679@shell01 HW 6]$ getconf LEVEL1_DCACHE_SIZE
32768
[G25866679@shell01 HW 6]$ getconf LEVEL1_DCACHE_LINESIZE
64
[G25866679@shell01 HW 6]$ getconf LEVEL2_CACHE_SIZE
1048576
[G25866679@shell01 HW 6]$ getconf LEVEL3_CACHE_SIZE
20185088
[G25866679@shell01 HW 6]$ cat /proc/cpuinfo | grep "model name" | head -1
model name      : Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz
```

Understanding rotate: rotate the image 90° and store image in dst.img. The image is transposed first and then rows are exchanged.

Transpose: $(i, j) \leftrightarrow (j, i)$

Exchange rows: row $i \leftrightarrow$ row $N-1-i$ (e.g. row 0 swaps with last row)

```
1. for (j = 0; j < dim; j++)
2.   for (i = 0; i < dim; i++)
3.     dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
```

- reading is in column-major order
- writing into dst is in row-major order
- more memory-intensive than smooth

Understanding smooth: return the smooth image in dst by taking the average of all pixels around a pixel value (maximum is $3 \times 3 = 9$ pixels).

```

1. for (j = 0; j < dim; j++)
2.   for (i = 0; i < dim; i++)
3.     dst[RIDX(i, j, dim)] = avg(dim, i, j, src);

```

- computationally more intensive because of memory access
 - must keep track of pixels
 - more compute-intensive than rotate

Optimizing Rotate

Techniques Attempted and Analysis

****NOTE: EVERY RUN PRODUCES DIFFERENT PERFORMANCE RESULTS****

Initial rotate

Testing Rotate:				
	Time in milliseconds		Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	880	964	2545982	2508741
1024	6231	6197	16224974	16110830
2048	44362	42514	115228755	110411215
4096	768965	739483	1996599716	1920008807

Initial performance for rotate without optimizations

```

1. for (j = 0; j < dim; j++)
2.   for (i = 0; i < dim; i++)
3.     dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];

```

Loop interchange

This is the process of exchanging the inner and outer loops to ensure that elements of a multidimensional array are accessed in order to improve locality of reference. Since a processor retrieves an entire block of data from memory to cache, there are fewer cache misses if the block consists of contiguously accessed array elements. Hence why accessing arrays by row is better than by columns in this case.

I thought that interchanging the loops would make writing into dst cheaper because reading src would then be faster due to row-major access. There would be less time spent in accessing src and less cache misses, meaning the waiting time for write would be less.

The rotate code is traversing in column-major order but laying out src in row-major order. To rewrite:

```

1. for (i = 0; i < dim; i++)
2.   for (j = 0; j < dim; j++)

```

```
3.    dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
```

From the performance results, we observe that loop interchange did not improve the performance; it actually hurt the performance. This is because dst is accessed column-wise while src is accessed row-wise, so the program jumps between different rows, increasing cache misses and slowing down writes.

I realized that the performance worsened because writing is cheaper before interchanging. This is due to how pixels are accessed. When transposing, we want to do so for a single pixel, not an entire cache line. Since the pixels we are transposing are not in a single cache line, it is better to access them by columns so we get fewer misses. After interchanging, writing is more expensive because the read takes more time + cache misses due to row-major access. As a result, dst takes more time since it has to write pixels that aren't in the same cache line.

Before interchange:

- src[RIDX(i, j)]: accesses (0,0), (1,0), (2,0)..
 - column-major -> bad locality
- dst[RIDX(i, j)]: accesses (0,0), (0,1), (0,2)..
 - row-major -> good locality

After interchange:

- src[RIDX(i, j)]: accesses (0,0), (0,1), (0,2)..
 - row-major -> good locality
- dst[RIDX(i, j)]: accesses (2,0), (1,0), (0,0)..
 - column-major -> bad locality

Why performance got worse:

- Every time the program reads a new cache line, the CPU must read it from memory
- Larger dimensions = more reads from memory

Solution:

- Need to improve locality for both source and destination -> blocking

Testing Rotate:				
Time in milliseconds			Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	980	986	2552135	2564651
1024	6446	11612	16755680	30159636
2048	46963	126057	124212303	327393176
4096	761812	1956442	1978472026	5080753737

After loop interchange

Dimension	Initial Speedup	% improved	Optimized Speedup	% improved	Improved?
512	880/964 = 0.9128	-9.5455%	980/986 = 0.9939	-0.6122%	No
1024	6231/6197 = 1.0054	0.5457%	6446/11612 = 0.5551	-80.1427%	No
2048	44362/42514 = 1.0434	4.1657%	46963/126057 = 0.3725	-168.4176%	No
4096	768965/739483 = 1.0398	3.8333%	761812/1956442 = 0.3893	-156.8142%	No

Loop nest optimization

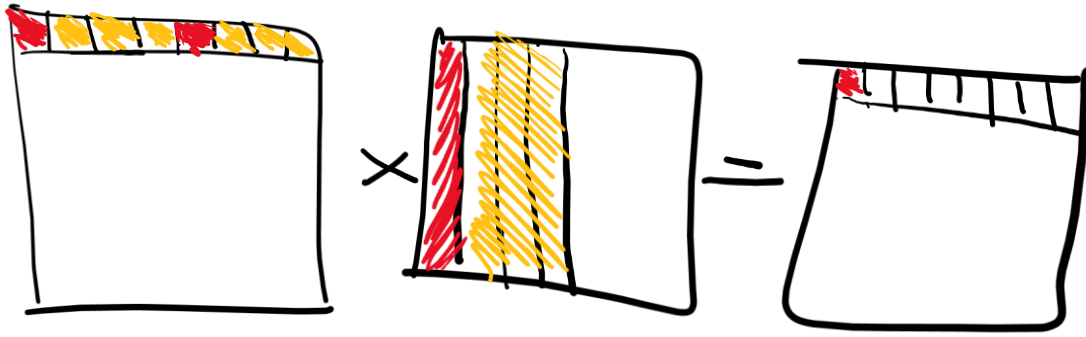
Since we are trying to access pixels in a block, not in a cache line, we must do blocking. Loop blocking is a technique that partitions large blocks into smaller ones (to fit into the caches) to achieve fewer cache misses.

I wanted to achieve fewer cache misses and evictions by accessing pixels we only need. The correct approach is blocking because we are trying to access pixels in a block, not a cache line. Without blocking, the src array will be evicted after the inner loop ends, if the dst array is large enough. It is a waste calling an entire column from dst array if we are only using one entry. With blocking, we improve temporal locality because we decrease the size of the data the CPU must store. Blocking segments array access into squares so that the CPU doesn't have to evict data since there is space in the cache. So, by iterating inside a block, we have less cache misses since we can reuse data.

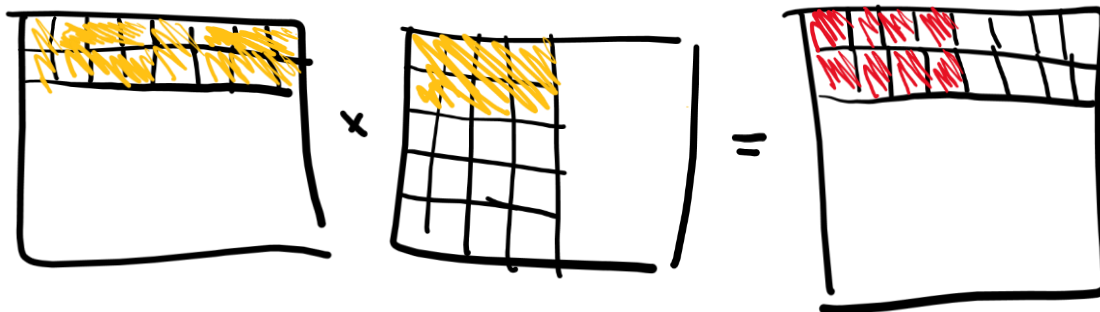
```

1. #ifndef min
2. #define min(a,b) (a < b ? a : b)
3. #endif
4. #define TILE_SIZE 32
5.
6. int ii, jj;
7.
8.     for (ii = 0; ii < dim; ii += TILE_SIZE)
9.         for (jj = 0; jj < dim; jj += TILE_SIZE)
10.            for (i = ii; i < min(dim, ii + TILE_SIZE); i++)
11.                for (j = jj; j < min(dim, jj + TILE_SIZE); j++)
12.                    dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];

```



Before blocking and transpose, the src is being read in row-major order and dst is being written in column-major order. Reads to rows are slower because for each cache element in a cache line, an element in a column must be accessed, so access is not contiguous. This scenario creates cache thrashing/pollution because every element in each row is computed with the dot product of every single column (entire matrix is read). Data is constantly being evicted.



After blocking and transpose, elements are stored in both rows and columns such that data is accessed by `[row][col]` for each element in the cache line. There are fewer cache misses because we only read the rows and columns we need. We are still reading src in row-major and writing dst in column-major, but blocking improves temporal locality, especially as the dimension gets bigger, because it allows the program to reuse elements multiple times before evicting it. Basically, once a block is loaded into cache, the block of data undergoes multiple operations within the loops. In addition, blocking becomes more effective when the dimension increases since the data size exceeds the cache size. For example, a 512x512 matrix can hold 262144 total elements and since RGB values have 16-bit representations, each element is $3 \times 16 = 6$ bytes, then the total size in memory is 262144x6 bytes, but as the dimension gets larger, the space needed in memory also gets larger, which means not all the data can fit in the cache.

Block size of 32 maximizes cache utilization. There is a balance between cache reuse and loop overhead, as it's not overly small or large to create cache pollution. 32x32 blocks also fit in L1 cache because $32 \times 32 \times 6 = 6144$ bytes < 32768 bytes L1 cache.

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      943      984      2451725      2559341
1024     6176     6868     16043597     17840344
2048     41026    38099     106538948    98968685
4096     738682   195018     1917846547   506430806

```

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      1014      967      2648066      2514547
1024     5695     6861     14798036     17825444
2048     38194    37506     99227470     97445733
4096     733151   192830    1903904550   500873350

```

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      919      981      2390240      2553213
1024     5818     6675     15116509     17594378
2048     38896    38259     101045454    99423059
4096     729458   195891    1894261722   508849909

```

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      915      977      2383290      2541750
1024     5688     6809     14779329     17690227
2048     40192    38229     104398349    99324463
4096     741002   195350    1924180647   507474770

```

Couple of runs after loop blocking for block size 32

Dimension	Optimized speedup	% improved	Improved (comparing to initial)?	Improved (comparing to loop interchange)?
512	$943/984 = 0.9583$	-4.3478%	Yes	Yes
1024	$6176/6868 = 0.8992$	-11.2047	No	Yes
2048	$41026/38099 = 1.0768$	7.1345%	Yes	Yes
4096	$738682/195018 = 3.7877$	73.5992%	Yes	Yes

Speedups for block size 32 (first image)

Interchange inner loops

To improve further, we want to maximize data reuse before it is replaced.

```
1. #define TILE_SIZE 32
2.
3. int ii, jj;
4.
5. for (ii = 0; ii < dim; ii += TILE_SIZE)
6.     for (jj = 0; jj < dim; jj += TILE_SIZE)
7.         for (j = jj; i < min(dim, jj + TILE_SIZE); j++)
8.             for (i = ii; i < min(dim, ii + TILE_SIZE); i++)
9.                 dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
```

Again, we can interchange the two inner loops. As mentioned, src is accessed in row-major order and dst is accessed in column-major order. By interchanging, dst has better locality. We want better locality for dst because the code is memory-sensitive, so reducing cache misses and evictions is essential. Although the locality for src is worse with this loop interchange, it is much better that dst has better locality because writes are more expensive than reads in this case. To store a new memory address, the CPU loads a cache line. If writes are non-contiguous, the CPU will have to constantly bring in new cache lines and evict.

I found that a block size of 32 gives the best overall performance. If the block size is less than 32, the performance results for dimension 512 are terrible. This means block size of 16 is too small for temporal locality to be utilized. The 16x16 block does not get reused enough before the next block's data replaces it. However, as the block size increases over 32, the performance starts to gradually decrease for the larger dimensions (e.g. 1024, 2048, 4096) because of cache pollution. When the block becomes too large, data is evicted too soon, increasing cache misses.

For a 16x16 image:

- src = 16x16x6 = 3072 bytes
- dst = 16x16x6 = 3072 bytes
- 6144 < 32768 (L1 cache)
- Too small
- Underutilizes temporal locality

For a 32x32 image:

- src = 32x32x6 = 6144 bytes
- dst = 32x32x6 = 6144 bytes

- $12288 < 32768$ (L1 cache)
- Optimal as it utilizes cache reuse

For a 64x64 image:

- $\text{src} = 64 \times 64 \times 6 = 24576$ bytes
- $\text{dst} = 64 \times 64 \times 6 = 24576$ bytes
- $49152 > 32768$ (L1 cache)
- Cache thrashing because it exceeds L1 cache
- Too large
- More cache evictions as the image exceeds the cache capacity

Testing Rotate:

	Time in milliseconds		Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	935	1054	2436273	2740846
1024	6064	4140	15753799	10759495
2048	41800	26148	108578960	67979852
4096	771134	127989	2002847795	332549099

Dimension	Optimized speedup	% improved
512	$935/1054 = 0.8870$	-12.7273%
1024	$6064/4140 = 1.4647$	31.7282%
2048	$41800/26148 = 1.5985$	37.4449%
4096	$771134/127989 = 6.0250$	83.4025%

For block size 16

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      986      917      2565595      2399530
1024     5988     3815     15554914     9912951
2048     41438    28708    107626156    74565838
4096     772201    162147    2004962389    421056260

```

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      900      892      2338620      2317567
1024     5846     3778     15184510     9814626
2048     43674    28023    113425891    72774422
4096     772384    161149    2005408524    418441506

```

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      973      896      2532633      2330674
1024     6346     3850     16535162     10002583
2048     43129    28921    112105199    75210601
4096     794077    169155    2063305605    439460127

```

Dimension	Optimized speedup	% improvement	Improved (compared to initial)?	Improved (compared to loop interchange)?	Improved (compared to blocking only)?
512	973/896 = 1.0859	7.9137%	Yes	Yes	Yes
1024	6346/3850 = 1.6493	39.3319%	Yes	Yes	Yes
2048	43129/28921 = 1.4912	32.9430%	Yes	Yes	Yes
4096	794077/169155 = 4.6943	78.6979%	Yes	Yes	Yes

Comparing speedups for block size 32 (third image)

```

Testing Rotate:
      Time in milliseconds      Cycles used
=====
Dimension naive_rotate my_rotate  naive_rotate my_rotate
=====
512      927      979      2434118      2547843
1024     6204     3932     16137913     10229448
2048     41615    29382    108131144    76335718
4096     748374    169048    1943895810    439587110

```

Dimension	Optimized speedup	% improved
512	927/979 = 0.9468	-5.6095%
1024	6204/3932 = 1.5778	36.6215%
2048	41615/29382 = 1.4163	29.3957%
4096	748374/169048 = 4.4269	77.4113%

Speedups for block size 64

Testing Rotate:				
Time in milliseconds			Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	954	944	2482019	2454913
1024	6067	3802	15760405	9880094
2048	43957	29582	114193450	76874146
4096	784620	158989	2038105791	413077486

Dimension	Optimized speedup	% improved
512	954/944 = 1.0105	1.0482%
1024	6067/3802 = 1.5957	37.3331%
2048	43957/29582 = 1.4859	32.7024%%
4096	784620/158989 = 4.9350	79.7368%

Speedups for block size 96

Testing Rotate:				
Time in milliseconds			Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	1100	1031	2874261	2690042
1024	6084	3804	15809735	9883256
2048	39731	27902	103231894	72500143
4096	734771	172935	1908373414	449515771

Dimension	Optimized speedup	% improved
512	1100/1031 = 1.0669	6.2727%

1024	6084/3804 = 1.5993	37.4753%
2048	39731/27902 = 1.4239	29.7727%
4096	734771/172935 = 4.2488	76.4641%

Speedups for block size 128

Testing Rotate:				
Time in milliseconds			Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	880	951	2298498	2472371
1024	6158	4068	16002307	10576954
2048	40401	28721	105031200	74672547
4096	733766	184154	1907187197	478443648

Dimension	Optimized speedup	% improved
512	880/951 = 0.9253	-8.0682%
1024	6158/4068 = 1.5137	33.9396%
2048	40401/29721 = 1.3693	26.4349%
4096	733766/184154 = 3.9845	74.9029%

Speedups for block size 160

Loop unrolling

We can improve performance further by unrolling the instructions inside the loops. Without unrolling, we are doing the transpose one at a time and then for each transpose, the program must go through the loops repeatedly. This is a waste of space and time (e.g. increments, branches) since we know the instructions can be executed independently, as there are no loop-carried dependencies. With unrolling, we can have more instructions in parallel by scheduling and reduce branch overhead since we access it only TILE_SIZE/8 times.

I tried unrolling 4x, 8x, and 16x. Out of the three, unrolling 8x gave the best performance results.

Since dim is a multiple of 4, we can unroll successfully without any out of bounds error. To unroll, we can explicitly write lines for each iteration value. So, each iteration handles 4, 8, and 16 elements, which is better than just doing one per iteration. Unrolling allows the

compiler to exploit ILP to allow multiple independent instructions to execute simultaneously, keeping functional units busy and reducing stalls. In addition, throughput is higher because more data can be executed and written back in a time frame.

Since `dst = src` is traversing the array by row (e.g. `[0][0]`, `[0][1]`, `[0][2]`), we can just write the instructions for the next `x` instructions.

Unrolling 8x gave the best overall results compared to unrolling 4x and 16x. Between 4x and 8x, performance increased and between 8x and 16x, performance decreased for two of the dimensions.

As the unroll increases, there is better ILP but unrolling too many times increases instruction count and register pressure. More memory is consumed when instruction count increases, which can result in cache misses if the unrolled loops exceed the cache size. In addition, more registers are needed to hold the values. If we unroll too many times, we need more registers than available.

Why it works

1. Reduces loop overhead: every loop iteration requires bookkeeping by the CPU even though these operations don't contribute to the actual work. Without unrolling, for 32 pixels, you need 32 iterations. With unrolling 8x, for 32 pixels, you only need 4 (32/8) iterations, reducing the overhead. This allows the CPU to spend more time on the actual work of the program.
2. Enables ILP: with unrolling, the CPU can work on several loads and stores at the same time.
3. Keeps CPU busy: without unrolling, the loop creates dependencies that prevent units from being fully utilized. With unrolling, the CPU has enough instructions to keep all its execution units busy.

Why 4x unrolling is not enough

- The CPU units are under utilized
- There are some execution units that are idle

Why 16x unrolling is too much

- The compiler needs more registers to hold temporary values for all the operations
- When registers run out, the compiler must spill values to memory, which is slower

```
1. //try 4
2. int ii, jj;
3. for (ii = 0; ii < dim; ii += TILE_SIZE)
4.     for (jj = 0; jj < dim; jj += TILE_SIZE)
5.         for (j = jj; j < min(dim, jj + TILE_SIZE); j++)
```

```

6.     for (i = ii; i < min(dim, ii + TILE_SIZE); i += 4) {
7.         dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
8.         dst[RIDX(dim-1-j, i+1, dim)] = src[RIDX(i+1, j, dim)];
9.         dst[RIDX(dim-1-j, i+2, dim)] = src[RIDX(i+2, j, dim)];
10.        dst[RIDX(dim-1-j, i+3, dim)] = src[RIDX(i+3, j, dim)];
11.    }
12.
13.    //try 8
14.    int ii, jj;
15.    for (ii = 0; ii < dim; ii += TILE_SIZE)
16.        for (jj = 0; jj < dim; jj += TILE_SIZE)
17.            for (j = jj; j < min(dim, jj + TILE_SIZE); j++)
18.                for (i = ii; i < min(dim, ii + TILE_SIZE); i += 8) {
19.                    dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
20.                    dst[RIDX(dim-1-j, i+1, dim)] = src[RIDX(i+1, j, dim)];
21.                    dst[RIDX(dim-1-j, i+2, dim)] = src[RIDX(i+2, j, dim)];
22.                    dst[RIDX(dim-1-j, i+3, dim)] = src[RIDX(i+3, j, dim)];
23.                    dst[RIDX(dim-1-j, i+4, dim)] = src[RIDX(i+4, j, dim)];
24.                    dst[RIDX(dim-1-j, i+5, dim)] = src[RIDX(i+5, j, dim)];
25.                    dst[RIDX(dim-1-j, i+6, dim)] = src[RIDX(i+6, j, dim)];
26.                    dst[RIDX(dim-1-j, i+7, dim)] = src[RIDX(i+7, j, dim)];
27.                }
28.
29.    //try 16
30.    int ii, jj;
31.    for (ii = 0; ii < dim; ii += TILE_SIZE)
32.        for (jj = 0; jj < dim; jj += TILE_SIZE)
33.            for (j = jj; j < min(dim, jj + TILE_SIZE); j++)
34.                for (i = ii; i < min(dim, ii + TILE_SIZE); i += 16) {
35.                    dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
36.                    dst[RIDX(dim-1-j, i+1, dim)] = src[RIDX(i+1, j, dim)];
37.                    dst[RIDX(dim-1-j, i+2, dim)] = src[RIDX(i+2, j, dim)];
38.                    dst[RIDX(dim-1-j, i+3, dim)] = src[RIDX(i+3, j, dim)];
39.                    dst[RIDX(dim-1-j, i+4, dim)] = src[RIDX(i+4, j, dim)];
40.                    dst[RIDX(dim-1-j, i+5, dim)] = src[RIDX(i+5, j, dim)];
41.                    dst[RIDX(dim-1-j, i+6, dim)] = src[RIDX(i+6, j, dim)];
42.                    dst[RIDX(dim-1-j, i+7, dim)] = src[RIDX(i+7, j, dim)];
43.                    dst[RIDX(dim-1-j, i+8, dim)] = src[RIDX(i+8, j, dim)];
44.                    dst[RIDX(dim-1-j, i+9, dim)] = src[RIDX(i+9, j, dim)];
45.                    dst[RIDX(dim-1-j, i+10, dim)] = src[RIDX(i+10, j, dim)];
46.                    dst[RIDX(dim-1-j, i+11, dim)] = src[RIDX(i+11, j, dim)];
47.                    dst[RIDX(dim-1-j, i+12, dim)] = src[RIDX(i+12, j, dim)];
48.                    dst[RIDX(dim-1-j, i+13, dim)] = src[RIDX(i+13, j, dim)];
49.                    dst[RIDX(dim-1-j, i+14, dim)] = src[RIDX(i+14, j, dim)];
50.                    dst[RIDX(dim-1-j, i+15, dim)] = src[RIDX(i+15, j, dim)];
51.                }

```

Testing Rotate:

	Time in milliseconds		Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	888	726	2314186	1892846
1024	7723	4272	20086322	11113175
2048	43857	23327	114014731	60645798
4096	739011	136198	1919080284	353871396

Dimension	Optimized speedup	% improved
-----------	-------------------	------------

512	888/726 = 1.2231	18.2432%
1024	7723/4272 = 1.8078	44.6847%
2048	43857/23327 = 1.8800	46.810%%
4096	739011/136198 = 5.4260	81.5702%

Unrolled 4x

Testing Rotate:				
Time in milliseconds			Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	918	720	2399790	1873895
1024	6099	3134	15858504	8413757
2048	42052	20998	109249772	54598605
4096	742048	126810	1927240232	329687543

Dimension	Optimized speedup	% improved
512	918/720 = 1.275	21.5686%
1024	6099/3134 = 1.9460	48.6145%
2048	42052/20998 = 2.0026	50.0666%
4096	742048/126810 = 5.8516	82.9108%

Unrolled 8x

Testing Rotate:				
Time in milliseconds			Cycles used	
Dimension	naive_rotate	my_rotate	naive_rotate	my_rotate
512	934	742	2436866	1931117
1024	6069	2894	15768757	7522831
2048	40646	22176	105661625	57666920
4096	758691	111533	1970447350	290120029

Dimension	Optimized speedup	% improved
512	934/742 = 1.2587	20.5567%
1024	6069/2894 = 2.0971	52.3150%
2048	40646/22176 = 1.8328	45.4411%

4096	758691/111533 = 6.8023	85.2993%
------	---------------------------	----------

Unrolled 16x

Optimizing Smooth

Techniques Attempted and Analysis

****NOTE: EVERY RUN PRODUCES DIFFERENT PERFORMANCE RESULTS****

Initial smooth

```
1. for (j = 0; j < dim; j++)
2.   for (i = 0; i < dim; i++)
3.     dst[RIDX(i, j, dim)] = avg(dim, i, j, src);
```

```
1. /*
2.  * avg - Returns averaged pixel value at (i,j)
3.  */
4. static pixel avg(int dim, int i, int j, pixel *src)
5. {
6.   int ii, jj;
7.   pixel_sum sum;
8.   pixel current_pixel;
9.
10.  initialize_pixel_sum(&sum);
11.  for(ii = maximum(i-1, 0); ii <= minimum(i+1, dim-1); ii++)
12.    for(jj = maximum(j-1, 0); jj <= minimum(j+1, dim-1); jj++)
13.      accumulate_sum(&sum, src[RIDX(ii, jj, dim)]);
14.
15.  assign_sum_to_pixel(&current_pixel, sum);
16.  return current_pixel;
17. }
```

```
1. /*
2.  * initialize_pixel_sum - Initializes all fields of sum to 0
3.  */
4. static void initialize_pixel_sum(pixel_sum *sum)
5. {
6.   sum->red = sum->green = sum->blue = 0;
7.   sum->num = 0;
8.   return;
9. }
```

```
1. /*
2.  * assign_sum_to_pixel - Computes averaged pixel value in current_pixel
3.  */
4. static void assign_sum_to_pixel(pixel *current_pixel, pixel_sum sum)
5. {
6.   current_pixel->red = (unsigned short) (sum.red/sum.num);
7.   current_pixel->green = (unsigned short) (sum.green/sum.num);
8.   current_pixel->blue = (unsigned short) (sum.blue/sum.num);
9.   return;
```

```
10. }
```

```
1. /*
2.  * accumulate_sum - Accumulates field values of p in corresponding
3.  * fields of sum
4.  */
5. static void accumulate_sum(pixel_sum *sum, pixel p)
6. {
7.     sum->red += (int) p.red;
8.     sum->green += (int) p.green;
9.     sum->blue += (int) p.blue;
10.    sum->num++;
11.    return;
12. }
```

```
Testing Smooth:
      Time in milliseconds      Cycles used
=====
Dimension naive_smooth my_smooth  naive_smooth my_smooth
=====
256      6766      6753      17568265      17535734
512     29059     29015     75458648     75350885
1024    127371    127640    330737483    331446625
2048    690892    710249    1793977616    1845070007
```

```
Testing Smooth:
      Time in milliseconds      Cycles used
=====
Dimension naive_smooth my_smooth  naive_smooth my_smooth
=====
256      6644      6644      17250323      17246927
512     29016     28980     75330859     75236947
1024    128952    126602    334830753    328682915
2048    705580    701172    1831936264    1820464040
```

```
Testing Smooth:
      Time in milliseconds      Cycles used
=====
Dimension naive_smooth my_smooth  naive_smooth my_smooth
=====
256      6639      6637      17251968      17242124
512     28658     28817     74461840     74874645
1024    125958    126472    327256385    328594480
2048    669780    676416    1739153066    1756355613
```

Initial performances for smooth without optimization

Dimension	Initial Speedup	% improved
256	6639/6637 = 1.0003	0.03012%
512	28658/28817 = 0.9944	-0.5548%

1024	125958/126472 = 0.9959	-0.4081%
2048	669780/676416 = 0.9901	-0.9908%

Speedup table for third image

Loop interchange

Since the smooth function is more computationally intensive, having faster access to avg will reduce the writeback of dst, meaning that there are less cycles used. However, we notice that avg is a function call (will address in inlining section), meaning that the dst will have to wait until avg is finished before writing back.

Like for the rotate function, we can interchange the loops for the smooth function. There is an improvement because the program is accessing contiguous elements. Also, dst (writing) and avg (reading) are iterating over columns to access memory contiguously. Before interchange, dst and src within avg is accessed column-wise. After interchange, dst and src have better cache reuse because of row-major access.

Why it works

1. Better spatial locality: before interchange, writing to dst in column-major order means each write jumps to a different row, which requires a new cache line to be loaded. Since only one pixel is used before jumping to a completely different memory location, this is wasteful. After interchange, writing to dst in row-major order means consecutive writes use the same cache line. When you load a cache line, all the data are used before moving to the next cache line.

```
1. for (i = 0; i < dim; i++)
2.   for (j = 0; j < dim; j++)
3.     dst[RIDX(i, j, dim)] = avg(dim, i, j, src);
```

Testing Smooth:				
Time in milliseconds			Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6736	6552	17494480	17020224
512	28275	26840	73422017	69679300
1024	127068	107282	329961274	278540221
2048	709304	430198	1841652810	1116872607

```

Testing Smooth:
      Time in milliseconds      Cycles used
=====
Dimension naive_smooth my_smooth  naive_smooth my_smooth
=====
256      6812      6516      17699262      16932926
512      28871      26700      74977888      69331992
1024     127226     107924      330462631     280299034
2048     717893     446861      1864782903     1161404752

```

```

Testing Smooth:
      Time in milliseconds      Cycles used
=====
Dimension naive_smooth my_smooth  naive_smooth my_smooth
=====
256      6643      6491      17253247      16860533
512      28808      26745      74826747      69461383
1024     125054     107382      324841757     278856386
2048     722481     434708      1876611858     1129217662

```

Couple of runs after loop interchange

Dimension	Optimized speedup	% improvement	Improved (compared to initial)?
256	6643/6491 = 1.0234	2.2881%	Yes
512	28808/26745 = 1.0771	7.1612%	Yes
1024	125054/107382 = 1.1645	14.1315%	Yes
2048	722481/434708 = 1.6619	39.8312%	Yes

Speedups for the third image

Function inlining

This optimization technique replaces function calls with the body of the function. For instance, in the initial smooth function, the avg function is called inside it. However, with inlining, we can call the components of the avg function inside smooth function.

We can modify smooth by breaking the avg function. The avg function has three other function calls: initialize_pixel_sum, accumulate_sum, and assign_sum_to_pixel. By looking at the structure of the functions, we just rewrite it inside smooth.

Why inlining improves performance

1. Eliminates function call overhead: for every function call (number of instructions in machine code), the CPU must do bookkeeping, where it must save the current state, push parameters onto the stack, then call procedure. When the function is called

several times, the overhead becomes massive, and since the overhead does not do any real “work” of the function, we can just execute directly without the function call.

2. Enables ILP: function calls prevent ILP because the CPU must wait for the function to complete before continuing, which forces sequential execution. With inlining, the compiler can see all the operations and executes them at once, increasing throughput.

```

1. for (i = 0; i < dim; i++) {
2.     for (j = 0; j < dim; j++) {
3.         int red_sum = 0, green_sum = 0, blue_sum = 0;
4.         int num = 0;
5.
6.         int ii, jj;
7.         for (ii = maximum(i-1, 0); ii <= minimum(i+1, dim-1); ii++) {
8.             for (jj = maximum(j-1, 0); jj <= minimum(j+1, dim-1); jj++) {
9.                 pixel p = src[RIDX(ii, jj, dim)];
10.                red_sum += (int) p.red;
11.                green_sum += (int) p.green;
12.                blue_sum += (int) p.blue;
13.                num++;
14.            }
15.        }
16.
17.        pixel *new_pixel = &dst[RIDX(i, j, dim)];
18.        new_pixel->red = red_sum/num;
19.        new_pixel->green = green_sum/num;
20.        new_pixel->blue = blue_sum/num;
21.    }
22. }

```

Testing Smooth:				
	Time in milliseconds		Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6656	4575	17297805	11888850
512	28909	19035	75146156	49491308
1024	126669	75723	329256219	196785168
2048	704421	303536	1831192736	789028839

Testing Smooth:				
	Time in milliseconds		Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6629	4574	17225777	11886753
512	28743	18816	74711467	48898795
1024	125189	75735	325395557	196816953
2048	685234	304984	1781349769	792865982

Testing Smooth:				
	Time in milliseconds		Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6622	4573	17211126	11884357
512	28463	19149	73976728	49767402
1024	124452	75515	323498435	196260328
2048	712439	303097	1852164432	787886732

Dimension	Optimized speedup	% improvement	Improved (compared to loop interchange)?
256	6622/4573 = 1.4480	30.9423%	Yes
512	28463/19149 = 1.4864	32.7232%	Yes
1024	124452/75515 = 1.6480	39.3219%	Yes
2048	712439/303097 = 2.3505	57.4564%	Yes

Speedups using image 3 (from above)

Loop blocking

Since blocking helped the performance for rotate, I thought combining it with inlining would help for smooth. However, blocking did not benefit the performance and was worse compared to just inlining.

```

1. int ii, jj, iii, jjj;
2. for (ii = 0; ii < dim; ii += TILE_SIZE) {
3.     for (jj = 0; jj < dim; jj += TILE_SIZE) {
4.         for (i = ii; i < minimum(ii + TILE_SIZE, dim); i++) {
5.             for (j = jj; j < minimum(jj + TILE_SIZE, dim); j++) {
6.                 int red_sum = 0, green_sum = 0, blue_sum = 0;
7.                 int num = 0;
8.
9.                 for (iii = maximum(i-1, 0); iii <= minimum(i+1, dim-1); iii++) {
10.                    for (jjj = maximum(j-1, 0); jjj <= minimum(j+1, dim-1); jjj++) {
11.                        pixel p = src[RIDX(iii, jjj, dim)];
12.                        red_sum += (int) p.red;
13.                        green_sum += (int) p.green;
14.                        blue_sum += (int) p.blue;
15.                        num++;
16.                    }
17.                }
18.
19.                pixel *new_pixel = &dst[RIDX(i, j, dim)];
20.                new_pixel->red = red_sum/num;
21.                new_pixel->green = green_sum/num;
22.                new_pixel->blue = blue_sum/num;

```

```

23.     }
24.   }
25. }
26. }

```

Since smooth is computationally-intensive compared to rotate, blocking is less effective. One of the main benefits of blocking is improving temporal locality by reusing cache, which is important to memory-intensive tasks like the rotate function.

Why blocking hurt performance

1. Added loop overhead: blocking introduced two additional outer loops (ii and jj), which means there are more branch instructions. Also, there are more minimum() function calls for every single pixel, even for interior pixels.
2. Reduced ILP: the extra loops break the computation into smaller segments. The CPU must manage more loop counters and branch predictions, leaving fewer resources for actual work.

Testing Smooth:				
	Time in milliseconds		Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	7086	4807	18410747	12483776
512	29161	20560	75738949	53399907
1024	131205	83755	340745772	217512062
2048	735878	368147	1911082001	956074908

Testing Smooth:				
	Time in milliseconds		Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6671	4834	17322325	12554321
512	28931	20278	75127131	52663435
1024	130053	82171	337764179	213393428
2048	683968	360609	1776295923	936496016

Testing Smooth:				
	Time in milliseconds		Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6761	4850	17558283	12596008
512	29598	20404	76868331	52981009
1024	131455	82006	341442167	212958709
2048	724134	365836	1880617070	950076833

Couple of runs after blocking

Dimension	Optimized speedup	% improvement	Improved (compared to inlining)?
256	6761/4850 = 1.3940	28.2650%	No
512	29598/20404 = 1.4506	31.0629%	No
1024	131455/82006 = 1.6029	37.6167%	No
2048	724134/365836 = 1.9794	49.4795%	No

Speedups using image 3 (from above)

Loop unrolling and loop splitting

To improve parallelism, we can apply loop unrolling and loop splitting.

Loop splitting is a technique to simplify a loop by breaking it down into multiple loops that have the same body but iterate over different portions.

We observe that the smooth function can be broken down like so:

```

1. for (i = 1; i < dim-1; i++) {
2.     for (j = 1; j < dim-1; j++) {
3.         pixel *new_pixel = &dst[RIDX(i, j, dim)];
4.         int red_sum = 0, green_sum = 0, blue_sum = 0;
5.
6.         red_sum += src[RIDX(i-1, j-1, dim)].red;
7.         green_sum += src[RIDX(i-1, j-1, dim)].green;
8.         blue_sum += src[RIDX(i-1, j-1, dim)].blue;
9.
10.        red_sum += src[RIDX(i-1, j, dim)].red;
11.        green_sum += src[RIDX(i-1, j, dim)].green;
12.        blue_sum += src[RIDX(i-1, j, dim)].blue;
13.
14.        red_sum += src[RIDX(i-1, j+1, dim)].red;
15.        green_sum += src[RIDX(i-1, j+1, dim)].green;
16.        blue_sum += src[RIDX(i-1, j+1, dim)].blue;
17.
18.        red_sum += src[RIDX(i, j-1, dim)].red;
19.        green_sum += src[RIDX(i, j-1, dim)].green;
20.        blue_sum += src[RIDX(i, j-1, dim)].blue;
21.
22.        red_sum += src[RIDX(i, j, dim)].red;
23.        green_sum += src[RIDX(i, j, dim)].green;
24.        blue_sum += src[RIDX(i, j, dim)].blue;
25.
26.        red_sum += src[RIDX(i, j+1, dim)].red;
27.        green_sum += src[RIDX(i, j+1, dim)].green;
28.        blue_sum += src[RIDX(i, j+1, dim)].blue;
29.
30.        red_sum += src[RIDX(i+1, j-1, dim)].red;
31.        green_sum += src[RIDX(i+1, j-1, dim)].green;
32.        blue_sum += src[RIDX(i+1, j-1, dim)].blue;
33.
34.        red_sum += src[RIDX(i+1, j, dim)].red;

```

```

35.     green_sum += src[RIDX(i+1, j, dim)].green;
36.     blue_sum += src[RIDX(i+1, j, dim)].blue;
37.
38.     red_sum += src[RIDX(i+1, j+1, dim)].red;
39.     green_sum += src[RIDX(i+1, j+1, dim)].green;
40.     blue_sum += src[RIDX(i+1, j+1, dim)].blue;
41.
42.     new_pixel->red = red_sum/9;
43.     new_pixel->green = green_sum/9;
44.     new_pixel->blue = blue_sum/9;
45. }
46. }
47.
48. for (i = 0; i < dim; i++) {
49.     for (j = 0; j < dim; j++) {
50.         if (i > 0 && i < dim-1 && j > 0 && j < dim-1)
51.             continue;
52.
53.         pixel *new_pixel = &dst[RIDX(i, j, dim)];
54.         int red_sum = 0, green_sum = 0, blue_sum = 0;
55.         int num = 0;
56.
57.         int ii, jj;
58.         for (ii = maximum(i-1, 0); ii <= minimum(i+1, dim-1); ii++) {
59.             for (jj = maximum(j-1, 0); jj <= minimum(j+1, dim-1); jj++) {
60.                 red_sum += src[RIDX(ii,jj,dim)].red;
61.                 green_sum += src[RIDX(ii,jj,dim)].green;
62.                 blue_sum += src[RIDX(ii,jj,dim)].blue;
63.                 num++;
64.             }
65.         }
66.
67.         new_pixel->red = red_sum/num;
68.         new_pixel->green = green_sum/num;
69.         new_pixel->blue = blue_sum/num;
70.     }
71. }

```

The smooth function averages the pixels in a maximum of 3x3 window. So, we can unroll 9x to calculate the pixel values for all 9 in one iteration. We also observe that the double inner loops (ii and jj) after inlining do boundary checking when they are not needed for non-edge pixels, which adds extra instructions. To fix this, we can first compute the avg values for the non-edge pixels by iterating from 1 to dim-1. Then we can compute only the edge pixels and skip the interior pixels since the program has already computed them. Previously, the smooth function went through the edge pixels and checked boundaries with the minimum and maximum functions, but now we can skip computing the interior pixels entirely and only compute the edge pixels.

This optimization reduced instruction overhead and increased ILP. Unrolling allowed the compiler to execute instructions in parallel so the CPU wouldn't have to wait on the computations. Splitting the loop eliminated redundant computation and nested loops, leading to less consumption of power by the CPU. The CPU spends less time performing tasks that don't contribute to the computation.

With only inlining, the program has more loop overhead and limited parallelism. It is iterating over the tile and also checking whether each pixel is an edge or not. Not only will there be more instructions, but there will also be more incorrect branch predictions, leading to pipeline flushes and stalls.

To reduce the cycles spent on control flow and branch predictions, we can first unroll the instruction to calculate for the whole 3x3 tile (only the 9 pixels in the center, not the edges). Then, we can split the loop to have a separate edge checker. If the pixel is not an edge, the loop continues until an edge pixel is found. If an edge pixel is found, the instructions after continue are executed and the valid pixels around the edge are averaged. The performance improved because there is less loop control overhead and allows the CPU to exploit ILP. The CPU spends less cycles issuing instructions, decoding, and executing. Also, splitting improves branch prediction accuracy because the loop that averages the interior has no branches. Splitting the loops helped a lot because checking edge pixel conditions hurt branch prediction accuracy since it is less predictable than loops.

Why this works

1. Eliminates redundant boundary checking: in the inlined version, every pixel performed boundary checks even though they were interior pixels. By splitting, we can eliminate overhead. Each boundary check had function calls to `maximum()` and `minimum()`, and other extra instructions, which can be avoided for interior pixels.
2. Reduces loop overhead: unrolling eliminates the inner loops because the CPU can just execute all the different positions in single lines.
3. Increases ILP: with unrolling, the CPU can load multiple pixels from memory simultaneously and perform multiple operations in parallel.

Testing Smooth:				
Time in milliseconds			Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6684	2635	17369991	6850389
512	29658	10645	77100005	27668560
1024	129271	42598	335677196	110611658
2048	710909	172531	1845875931	448020635

Testing Smooth:				
Time in milliseconds			Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6676	2600	17330349	6754089
512	29302	10585	76120747	27479899
1024	127563	42737	331169762	111001383
2048	714826	175324	1856018523	455659781

Testing Smooth:				
Time in milliseconds			Cycles used	
Dimension	naive_smooth	my_smooth	naive_smooth	my_smooth
256	6852	2599	17796284	6748114
512	28705	10594	74562664	27504478
1024	129850	42928	337679991	111569135
2048	730847	172256	1897915670	447220176

Dimension	Optimized speedup	% improvement	Improved (compared to inlining)?
256	$6852/2599 = 2.6362$	62.0695%	Yes
512	$28705/10594 = 2.7095$	63.0935%	Yes
1024	$129850/42928 = 3.0248$	66.9403%	Yes
2048	$730847/172256 = 4.2428$	76.4306%	Yes